

# UAB

## Universitat Autònoma de Barcelona

Universitat Autònoma de Barcelona

Grau en Sistemes Multimèdia · Curs 2025–2026

Hackathon — Informe del Projecte

# Dal-i

## Sistema Robòtic de Dibuix Col·laboratiu

Generació automàtica de trajectòries per a braç robòtic  
mitjançant visió per computador i intel·ligència artificial generativa



### Grup G4-6

| Nom i Cognoms          | NIU     |
|------------------------|---------|
| David Madueño Noguer   | 1526933 |
| Moisés Sánchez Pin     | 1603611 |
| Xavier Cañada Escalona | 1708948 |
| Lorenzo Payaro Robles  | 1731940 |

Juny 2026

# Índex

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducció</b>                                            | <b>2</b>  |
| 1.1      | Context . . . . .                                             | 2         |
| 1.2      | Motivació . . . . .                                           | 2         |
| 1.3      | Objectius . . . . .                                           | 2         |
| 1.4      | Estructura de l'informe . . . . .                             | 3         |
| <b>2</b> | <b>Descripció de l'Aplicació</b>                              | <b>3</b>  |
| 2.1      | Visió general . . . . .                                       | 3         |
| 2.2      | Flux d'ús . . . . .                                           | 3         |
| 2.3      | Funcionalitats principals . . . . .                           | 3         |
| 2.4      | Interfície d'usuari . . . . .                                 | 4         |
| <b>3</b> | <b>Arquitectura del Sistema</b>                               | <b>5</b>  |
| 3.1      | Visió general . . . . .                                       | 5         |
| 3.2      | Frontend . . . . .                                            | 5         |
| 3.3      | Cloud Run — Backend de Visió . . . . .                        | 5         |
| 3.4      | Cloud Functions . . . . .                                     | 6         |
| 3.5      | Emmagatzematge i Bases de Dades . . . . .                     | 6         |
| 3.6      | Autenticació . . . . .                                        | 7         |
| <b>4</b> | <b>Desenvolupament</b>                                        | <b>7</b>  |
| 4.1      | Fase 1 — Pipeline de Visió per Computador . . . . .           | 7         |
| 4.2      | Fase 2 — Desplegament a Cloud Run . . . . .                   | 8         |
| 4.3      | Fase 3 — Migració a Cloud Functions i Firebase Auth . . . . . | 8         |
| 4.4      | Fase 4 — Suport Multilingüe . . . . .                         | 9         |
| 4.5      | Fase 5 — Compatibilitat amb la Raspberry Pi . . . . .         | 9         |
| <b>5</b> | <b>Tecnologies Utilitzades</b>                                | <b>9</b>  |
| 5.1      | Serveis Cloud (Google Cloud Platform) . . . . .               | 9         |
| 5.2      | Tecnologies No-Cloud . . . . .                                | 10        |
| 5.3      | Organització del Codi Font . . . . .                          | 11        |
| <b>6</b> | <b>Problemes i Solucions</b>                                  | <b>12</b> |
| <b>7</b> | <b>Resultats</b>                                              | <b>14</b> |
| 7.1      | Sistema en Producció . . . . .                                | 14        |
| 7.2      | Pipeline de Visió . . . . .                                   | 14        |
| 7.3      | Exemples de Resultats . . . . .                               | 14        |
| 7.4      | Tests . . . . .                                               | 15        |
| 7.5      | Infraestructura Desplegada . . . . .                          | 16        |
| <b>8</b> | <b>Conclusions i Treball Futur</b>                            | <b>16</b> |
| 8.1      | Conclusions . . . . .                                         | 16        |
| 8.2      | Aprenentatges . . . . .                                       | 17        |
| 8.3      | Treball Futur . . . . .                                       | 17        |
|          | <b>Referències</b>                                            | <b>18</b> |

## 1 Introducció

### 1.1 Context

Aquest projecte s'ha desenvolupat en el marc del *hackathon* de l'assignatura de Sistemes Multimèdia del Grau en Sistemes Multimèdia de la Universitat Autònoma de Barcelona, durant el curs 2025–2026. L'objectiu del *hackathon* és dissenyar i implementar un sistema complet que integri múltiples tecnologies multimèdia i de computació al núvol en un termini limitat.

El projecte **Dal-i** —nom que fa referència a l'artista Salvador Dalí i al model d'IA generativa DALL-E— és un sistema que permet a qualsevol usuari crear dibuixos artístics a través d'un braç robòtic, sense necessitat de cap habilitat artística prèvia. El sistema accepta una imatge d'entrada i un estil artístic (introduïts per veu, text o selecció), processa la imatge mitjançant tècniques de visió per computador, i genera les coordenades de trajectòria que el robot ha de seguir per reproduir el dibuix.

### 1.2 Motivació

La motivació principal del projecte és construir un pont entre tres àmbits tecnològics que rarament es combinen en un sol sistema educatiu:

- **Intel·ligència artificial generativa:** ús de models de difusió i transformadors per generar o transferir estils artístics.
- **Visió per computador clàssica:** detecció de contorns, extracció de característiques i optimització de trajectòries per a robòtica.
- **Arquitectura cloud-native:** desplegament escalable i de baix cost mitjançant Google Cloud Platform, amb separació de responsabilitats en microserveis.

A més, l'addició de suport multilingüe (10 idiomes) i d'entrada per veu reflecteix l'interès del grup per fer el sistema accessible a usuaris de qualsevol llengua i entorn.

### 1.3 Objectius

Els objectius principals del projecte són:

1. Implementar un *pipeline* complet de processament d'imatges que converteixi qualsevol fotografia en coordenades de trajectòria per a un braç robòtic SCARA.
2. Desplegar el sistema sobre una arquitectura *cloud-native* escalable, fent ús de Cloud Run i Cloud Functions de Google Cloud Platform.
3. Oferir múltiples modalitats d'entrada: selecció d'estil per text, per veu i per botons de selecció ràpida.
4. Integrar IA generativa (Gemini 2.5 Flash) per a la transferència d'estil visual en un mode avançat opcional.
5. Donar suport multilingüe automàtic per permetre l'ús del sistema en 10 idiomes.
6. Proporcionar una experiència d'usuari fluida amb historial persistent per sessió, paginació i esborrat en cascada.

## 1.4 Estructura de l'informe

La resta d'aquest document s'organitza de la manera següent. La secció 2 descriu l'aplicació des del punt de vista de l'usuari. La secció 3 detalla l'arquitectura del sistema. La secció 4 explica el procés de desenvolupament per fases. La secció 5 llista les tecnologies utilitzades. La secció 6 recull els problemes trobats i les solucions adoptades. La secció 7 presenta els resultats obtinguts. Finalment, la secció 8 recull les conclusions i el treball futur.

## 2 Descripció de l'Aplicació

### 2.1 Visió general

Dal-i és una aplicació web que permet a l'usuari transformar qualsevol imatge en un dibuix artístic executat per un braç robòtic. El sistema actua com a intermediari entre la creativitat de l'usuari i la precisió mecànica del robot: l'usuari tria una imatge i un estil artístic, i el sistema calcula automàticament les trajectòries que el robot ha de seguir per reproduir el dibuix sobre paper.

### 2.2 Flux d'ús

El flux d'ús típic de l'aplicació és el següent:

1. L'usuari accedeix a la interfície web i selecciona una imatge d'entrada (fotografia, il·lustració, etc.).
2. L'usuari tria un estil artístic mitjançant una de les tres modalitats disponibles:
  - **Selecció ràpida:** botons predefinits (Picasso, Van Gogh, Monet, Manga, etc.).
  - **Text:** l'usuari escriu una instrucció en qualsevol dels 10 idiomes suportats.
  - **Veu:** l'usuari grava un comandament de veu; el sistema transcriu [1] i extreu l'estil automàticament.
3. Opcionalment, l'usuari pot activar el **Mode Avançat (IA Generativa)**, que usa Gemini 2.5 Flash [2] per aplicar la transformació visual a la imatge abans de la detecció de contorns.
4. El sistema processa la imatge i retorna:
  - Les **coordenades** de trajectòria per al robot (llista de punts  $\{x, y\}$ ).
  - Una **previsualització SVG** dels trazos generats.
  - Una **descripció artística** del resultat (i àudio TTS [3] en l'idioma de l'usuari).
5. La Raspberry Pi del braç robòtic rep les coordenades i executa el dibuix.
6. El resultat s'afegeix automàticament a l'**historial** de l'usuari, accessible en sessions posteriors.

### 2.3 Funcionalitats principals

**Entrada multimodal.** L'usuari pot interactuar per text, per veu (àudio WebM/Opus) o seleccionant un dels 25 estils artístics predefinits. Els tres modes coexisteixen en la mateixa interfície.

**Suport multilingüe (10 idiomes).** L'aplicació suporta espanyol, anglès, francès, alemany, italià, portuguès, català, japonès, xinès i àrab. El sistema tradueix automàticament [4] les

instruccions de l'usuari a l'anglès per al processament intern (extracció d'estil, models d'IA), i torna a traduir la resposta a l'idioma de l'usuari per a la confirmació TTS i el missatge de resultat.

**Mode Avançat amb Gemini.** Quan l'usuari activa el mode avançat, el sistema crida Vertex AI [2] amb el model Gemini 2.5 Flash per generar una versió de la imatge en l'estil artístic seleccionat. Aquesta imatge transformada és la que s'utilitza com a entrada per a la detecció de contorns, produint trazos molt més fidels a l'estil triat.

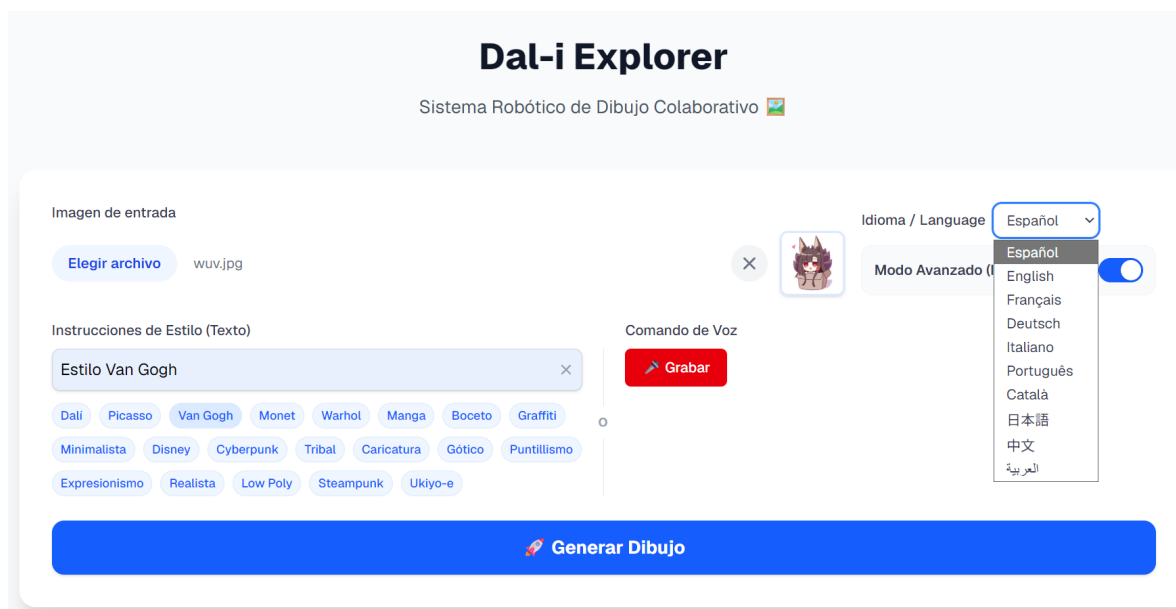
**Historial per sessió.** Cada navegador rep un identificador únic anònim (Firebase Anonymous Auth [5]) que permet mantenir un historial persistent de totes les generacions. L'historial es mostra paginat (12 elements per pàgina) i es pot esborrar parcialment (element individual) o totalment (botó «Eliminar tot» amb confirmació). L'esborrat en cascada elimina tant els registres de Firestore [6] com les imatges de Cloud Storage [7].

**Previsualització en temps real.** La interfície mostra la imatge original, la imatge transformada (si s'ha usat el mode avançat), i una superposició canvas dels trazos del robot sobre la imatge.

**Compatibilitat amb Raspberry Pi.** El backend ofereix els endpoints `/process/voice` i `/process/text` sense autenticació obligatòria, de manera que el maquinari pot cridar-los directament sense necessitat d'implementar Firebase Auth.

## 2.4 Interfície d'usuari

La interfície és una aplicació web responsiva construïda amb Next.js 16 [8] i React 19 [9], servida com a exportació estàtica des del mateix contenidor de Cloud Run [10]. El disseny és minimalista i funcional, amb un selector d'idioma, un selector d'estil (text + botons ràpids), un botó de gravació de veu i un interruptor per al mode avançat.



**Figura 1:** Interfície principal de Dal-i amb el selector d'idioma desplegat.

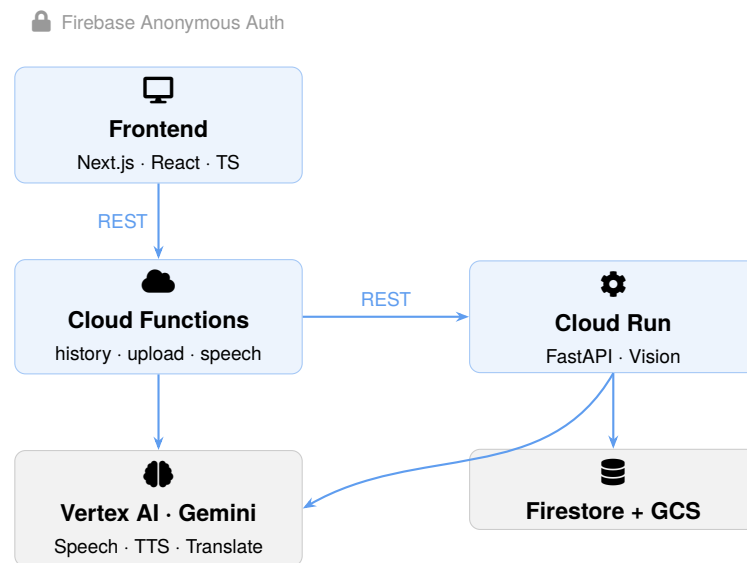
La **Figura 1** mostra l'estat de la interfície amb una imatge seleccionada, un estil introduït per text i el selector d'idioma desplegat, mostrant els 10 idiomes disponibles.

### 3 Arquitectura del Sistema

#### 3.1 Visió general

El sistema Dal-i segueix una arquitectura de microserveis *cloud-native* desplegada completament sobre Google Cloud Platform (GCP). La decisió de separar les operacions lleugeres (historial, càrrega d'imatges, parla i traducció) en Cloud Functions [11] independents, mantenint el pipeline de visió computacionalment intens en un servei Cloud Run [10] dedicat, permet escalar cada component de forma independent i reduir costos quan el sistema no està en ús.

La **Figura 2** mostra el diagrama d'arquitectura del sistema:



**Figura 2:** Diagrama d'arquitectura del sistema Dal-i sobre Google Cloud Platform.

#### 3.2 Frontend

El frontend és una aplicació **Next.js 16** [8] (React 19 [9], TypeScript, Tailwind CSS 4 [12]) compilada com a exportació estàtica (output: "export"). El bundle resultant es serveix directament des del contenidor Cloud Run, sense necessitat d'un servidor de renderització independent.

La comunicació amb el backend es centralitza al mòdul `app/lib/api.ts`, que s'encarrega d'injectar automàticament el token de Firebase Anonymous Auth [5] a totes les peticions com a capçalera `Authorization: Bearer <token>`. L'autenticació és completament transparent per a l'usuari: el SDK de Firebase crida `signInAnonymously()` en el primer muntatge del component i retorna un UID estable i persistent per al navegador.

#### 3.3 Cloud Run — Backend de Visió

El servei Cloud Run [10] (`dal-i-api`, regió `europa-west1`) és un contenidor Docker que executa una API REST amb FastAPI [13]. Conté el pipeline de processament d'imatges (computacionalment intens) i serveix els endpoints principals:

- `GET /health`: comprovació d'estat del servei.
- `POST /process`: pipeline complet per a selecció d'estil per botons.

- POST /process/voice: pipeline per a entrada de veu (compatible amb Raspberry Pi).
- POST /process/text: pipeline per a entrada de text (compatible amb Raspberry Pi).

Els endpoints /process requereixen un token de Firebase vàlid (o retornen uid="device" per a clients sense autenticació, com la Raspberry Pi). Tots els endpoints estan protegits per un limitador de velocitat en memòria (*sliding window*, 20 peticions/minut per IP).

El Dockerfile és multi-etapa: la primera etapa compila el frontend amb Node.js 20, i la segona copia el bundle estàtic i el codi Python al contenidor final basat en Python 3.12 slim.

### 3.4 Cloud Functions

Tres Cloud Functions [11] de 2a generació gestionen les operacions lleugeres:

history

Gestiona el CRUD de l'historial de generacions per usuari sobre Firestore [6]. Suporta:

- GET: llistat paginat (cursor basat en start\_after(snapshot), 12 elements per pàgina).
- DELETE ?id={docId}: esborra un element de Firestore i les imatges associades de Cloud Storage [7].
- DELETE ?deleteAll=true: esborra tots els elements de l'usuari en lots de fins a 500 (límit de batch de Firestore) i els blobs de GCS en cascada.

upload

Accepta una imatge per POST (multipart/form-data), la valida (màxim 10 MB, MIME image/\*) i la puja al bucket dal-i-bucket de Cloud Storage [7] amb accés públic. Retorna l'URL pública i el nom del fitxer.

speech

Agrupa tres operacions de parla i traducció en un sol endpoint amb paràmetre ?action=:

- transcribe: rep un fitxer d'àudio WebM/Opus, el transcriu amb Cloud Speech-to-Text [1] en l'idioma seleccionat, tradueix la transcripció a l'anglès (Cloud Translation [4]), i extreu l'estil artístic.
- tts: rep text en anglès i un codi d'idioma destí, tradueix el text a l'idioma de l'usuari, genera l'àudio MP3 amb Cloud TTS [3] (veus Journey o WaveNet) i el retorna en base64.
- translate: traducció de text simple entre qualsevol parell d'idiomes suportats.

Totes les Cloud Functions verifiquen el token de Firebase amb `firebase\_admin.auth().verify\_id\_token(token)` i afegeixen les capçaleres CORS necessàries per permetre crides des del domini de Cloud Run.

### 3.5 Emmagatzematge i Bases de Dades

**Cloud Storage** [7] (dal-i-bucket) emmagatzema les imatges d'entrada (images/), les imatges transformades per Gemini i els resultats JSON amb les coordenades (results/). Tots els blobs es fan públics en el moment de la pujada.

**Firestore** [6] emmagatzema els registres de l'història seguint la ruta `generation_history/{uid}/items/{docId}`, on `uid` és l'identificador anònim de Firebase. Cada registre inclou estil, URLs d'imatge, coordenades, descripció artística i marca de temps.

### 3.6 Autenticació

S'utilitza **Firestore Anonymous Authentication** [5]: quan un usuari accedeix a l'aplicació per primera vegada, el SDK de Firebase crea silenciosament un compte anònim i emmagatzema el token al `localStorage` del navegador. Aquest UID és estable entre sessions (mentre no s'esborri la memòria del navegador) i permet mantenir l'història personal sense que l'usuari hagi de registrar-se.

El backend verifica el token a totes les peticions; per als clients de maquinari (Raspberry Pi) que no implementen Firebase Auth, el sistema admet peticions sense token i les associa a l'UID genèric "device".

## 4 Desenvolupament

El desenvolupament del projecte s'ha organitzat en cinc fases progressives, cadascuna construint sobre l'anterior i afegint noves capacitats al sistema.

### 4.1 Fase 1 — Pipeline de Visió per Computador

El nucli del sistema és el pipeline de processament d'imatges implementat a `backend/src/vision.py`. Aquest pipeline transforma una imatge d'entrada en una llista de coordenades  $\{x, y\}$  que el braç robòtic pot seguir.

Detecció de contorns (Canny)

El primer pas és la detecció de vores amb l'algoritme de Canny [14] d'OpenCV [15]:

```

1 def detect_edges(image_bytes: bytes):
2     nparr = np.frombuffer(image_bytes, np.uint8)
3     img = cv2.imdecode(nparr, cv2.IMREAD_GRAYSCALE)
4     img = cv2.GaussianBlur(img, (5, 5), 0)
5     edges = cv2.Canny(img, threshold1=50, threshold2=150)
6     contours, _ = cv2.findContours(
7         edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE
8     )
9     return edges, img.shape[1], img.shape[0]
```

**Listing 1:** Detecció de vores amb Canny (`vision.py`)

Els paràmetres de Canny (llindars 50 i 150) s'han ajustat empíricament per obtenir un bon equilibri entre la captura de detalls i el soroll (vegeu **Codi 1**). Posteriorment, els contorns es simplifiquen amb `cv2.approxPolyDP` (`epsilon = 0.02`, àrea mínima = 20 píxels<sup>2</sup>) per reduir el nombre de punts i optimitzar el temps de dibuix del robot.

Optimització de trajectòries (TSP)

El repte principal del robot no és dibuixar els contorns, sinó *en quin ordre i en quina direcció* fer-ho, per minimitzar el temps de desplaçament amb el bolígraf aixecat. Hem implementat

una aproximació al *Travelling Salesman Problem* (TSP) [16] en tres passes:

1. **Greedy Nearest Neighbour** (`tsp_sort_contours`): ordena els contorns de manera voraç, triant sempre el contorn més proper al punt final del contorn anterior. Complexitat  $O(n^2)$ .
2. **Optimització de direccions** (`optimize_contour_directions`): per a cada contorn, decideix si recórrer-lo en l'ordre original o al revés, triant la direcció que minimitza la distància des del punt final del contorn anterior.
3. **2-opt local search** (`tsp_two_opt`): aplica fins a 3 passes d'intercanvi de parells de contorns per millorar la solució greedy. S'omet si hi ha més de 150 contorns per evitar temps de còmput excessius.

#### Transferència d'estil (Mode Avançat)

Quan l'usuari activa el Mode Avançat, el sistema crida `generate_styled_image()` que envia la imatge i el nom de l'estil a Vertex AI [2] amb el model **Gemini 2.5 Flash** en mode generació d'imatges. El model retorna una versió de la imatge en l'estil artístic sol·licitat, que s'utilitza com a entrada per a Canny en lloc de la imatge original. Aquesta imatge transformada també s'emmagatzema a Cloud Storage [7] i es mostra a l'usuari.

## 4.2 Fase 2 — Desplegament a Cloud Run

Un cop el pipeline de visió era funcional en local, l'hem empaquetat en un contenidor Docker multi-etapa i desplegat a Cloud Run [10]:

1. **Etapla 1 (Node.js 20)**: compila el frontend Next.js [8] amb `npm run build`, generant l'exportació estàtica a `/frontend/out`.
2. **Etapla 2 (Python 3.12 slim)**: instal·la les dependències Python (incloent OpenCV [15] i les biblioteques de GCP), copia el codi del backend i el bundle del frontend. L'arxiu `main.py` munta el directori `frontend/out` com a fitxers estàtics amb FastAPI [13] `StaticFiles`.

El desplegament automàtic s'ha configurat amb `cloudbuild.yaml`: en cada *push* a `main`, Cloud Build construeix la imatge, la puja a Artifact Registry i fa un rolling update del servei Cloud Run sense temps d'inactivitat.

## 4.3 Fase 3 — Migració a Cloud Functions i Firebase Auth

A mesura que el sistema creixia, vam identificar que les operacions lleugeres (historial, càrrega d'imatges) no necessitaven la memòria ni les dependències pesades (OpenCV [15], NumPy [17]) del contenidor de Cloud Run. La migració a Cloud Functions [11] redueix el cost de les operacions freqüents i permet escalar cada funció de forma independent.

En paral·lel, vam afegir **Firebase Anonymous Authentication** [5] per aïllar l'historial de cada usuari. El canvi d'esquema de Firestore [6] —de la col·lecció plana `generation_history/{docId}` a la subcol·lecció `generation_history/{uid}/items/{docId}`— va requerir actualitzar totes les funcions de `db.py` per acceptar el paràmetre `uid`.

La paginació de l'historial s'implementa amb el patró de cursor de Firestore: el client guarda l'ID del darrer document de la pàgina anterior i el passa com a `page_token`; el backend fa `.start_after(snapshot)` per obtenir la pàgina següent sense necessitat d'un OFFSET (que Firestore no suporta).

#### 4.4 Fase 4 — Suport Multilingüe

Per donar suport a 10 idiomes, vam integrar **Cloud Translation API** [4] a la Cloud Function speech. El disseny segueix el principi de *traducció interna*: el sistema sempre processa en anglès internament (perquè els noms dels estils artístics estan en anglès i els models d'IA funcionen millor en aquest idioma), i tradueix automàticament tant l'entrada de l'usuari (de l'idioma seleccionat a l'anglès) com la sortida (de l'anglès a l'idioma de l'usuari).

El flux de la modalitat de veu amb Cloud Speech-to-Text [1] i traducció és:

$$\text{Àudio (idioma X)} \xrightarrow{\text{STT}} \text{Text (idioma X)} \xrightarrow{\text{Translate}} \text{Text (anglès)} \xrightarrow{\text{extractStyle}} \text{Estil}$$

I el flux de la resposta amb Cloud TTS [3]:

$$\text{Missatge (anglès)} \xrightarrow{\text{Translate}} \text{Missatge (idioma X)} \xrightarrow{\text{TTS}} \text{Àudio (idioma X)}$$

Per evitar crides innecessàries a l'API de traducció quan l'idioma d'origen i el destí coincideixen (anglès → anglès), s'afegeix una comprovació prèvia: si `source_language.startswith("en")`, es retorna el text original sense traduir.

Les veus TTS s'han mapejat per a cada idioma, prioritzant les veus *Journey* (més naturals) i recorrent a *WaveNet* o *Standard* per als idiomes sense veu *Journey* disponible (català, xinès, àrab).

#### 4.5 Fase 5 — Compatibilitat amb la Raspberry Pi

Durant les proves d'integració, vam detectar que el braç robòtic (controlat per una Raspberry Pi) utilitzava l'endpoint `/process/voice` per enviar les imatges i l'àudio capturats localment. Quan vam migrar la parla a la Cloud Function, aquest endpoint va desaparèixer del Cloud Run, trencant la integració amb el maquinari.

La solució va ser restaurar `/process/voice` i `/process/text` al Cloud Run [10] amb una dependència d'autenticació opcional (`_optional_firebase_token`): si la petició no porta cap token, el sistema continua amb l'uid genèric "device" en comptes de retornar un error 401. Això permet que la Raspberry Pi operi sense implementar Firebase Auth [5].

### 5 Tecnologies Utilitzades

#### 5.1 Serveis Cloud (Google Cloud Platform)

La **Taula 1** recull tots els serveis GCP utilitzats al projecte.

**Taula 1:** Serveis GCP utilitzats al projecte Dal-i.

| Servei                    | Ús en el projecte                                                                                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cloud Run [10]            | Backend FastAPI + frontend estàtic. Executa el pipeline de visió, els endpoints <code>/process*</code> i serveix el bundle Next.js. Regió: <code>europa-west1</code> .            |
| Cloud Functions [11]      | Tres funcions lleugeres: <code>history</code> (CRUD Firestore), <code>upload</code> (pujada a GCS) i <code>speech</code> (STT, TTS, traducció). Regió: <code>us-central1</code> . |
| Vertex AI / Gemini [2]    | Transferència d'estil visual <code>image-to-image</code> en el Mode Avançat. Model: <code>gemini-2.5-flash</code> . Crida des de Cloud Run.                                       |
| Cloud Speech-to-Text [1]  | Transcripció d'àudio WebM/Opus a text. Suporta multidioma amb <code>alternative_language_codes</code> . Crida des de la CF <code>speech</code> .                                  |
| Cloud Text-to-Speech [3]  | Síntesi de veu (MP3) a partir de text. Veus Journey i WaveNet per a 10 idiomes. Crida des de la CF <code>speech</code> .                                                          |
| Cloud Translation API [4] | Traducció automàtica de text entre qualsevol parell dels 10 idiomes suportats. Crida des de la CF <code>speech</code> .                                                           |
| Cloud Storage [7]         | Emmagatzematge d'imatges d'entrada, imatges estilitzades i resultats JSON. Bucket: <code>dal-i-bucket</code> .                                                                    |
| Firestore [6]             | Base de dades documental per a l'històric per usuari. Col·lecció: <code>generation_history/{uid}/items</code> .                                                                   |
| Firebase Auth [5]         | Autenticació anònima del costat del client. UID persistent per navegador sense registre de l'usuari.                                                                              |
| Cloud Build               | CI/CD per a Cloud Run: construeix la imatge Docker, la puja a Artifact Registry i desplega el servei.                                                                             |
| Artifact Registry         | Registre privat d'imatges Docker del projecte.                                                                                                                                    |

## 5.2 Tecnologies No-Cloud

La **Taula 2** recull les tecnologies i biblioteques no-cloud del projecte.

**Taula 2:** Tecnologies no-cloud utilitzades al projecte.

| <b>Tecnologia</b>   | <b>Versió</b> | <b>Ús en el projecte</b>                                                                         |
|---------------------|---------------|--------------------------------------------------------------------------------------------------|
| Python              | 3.12          | Llenguatge principal del backend i les Cloud Functions.                                          |
| FastAPI [13]        | 0.110         | Framework REST del backend; gestió de rutes, validació Pydantic i middlewares CORS.              |
| OpenCV [15]         | 4.9           | Detecció de vores (Canny), extracció de contorns (findContours) i simplificació (approxPolyDP).  |
| NumPy [17]          | 1.26          | Operacions matricials en el processament d'imatges i càlcul de distàncies per a l'algoritme TSP. |
| Pillow              | 10.4          | Redimensionament i conversió de format de les imatges d'entrada.                                 |
| firebase-admin      | 6.5           | Verificació de tokens Firebase al backend (Cloud Run i Cloud Functions).                         |
| functions-framework | 3.8           | Execució de Cloud Functions Python tant en local com en producció.                               |
| httpx               | 0.27          | Client HTTP asíncron per a crides entre serveis interns.                                         |
| Next.js [8]         | 16.2          | Framework React per al frontend, compilat com a exportació estàtica.                             |
| React [9]           | 19.2          | Biblioteca d'interfície d'usuari amb hooks per a la gestió d'estat.                              |
| TypeScript          | 5.x           | Tipat estàtic del codi frontend per evitar errors en temps d'execució.                           |
| Tailwind CSS [12]   | 4.x           | Estils utilitaris per a la interfície d'usuari responsiva.                                       |
| Firebase SDK (JS)   | 10.x          | Autenticació anònima i obtenció de tokens al costat del client (navegador).                      |
| Docker              | —             | Contenidor multi-etapa per empaquetar backend Python i frontend estàtic.                         |
| pnpm                | 11.2          | Gestor de paquets Node.js amb gestió eficient del workspace.                                     |
| pytest              | 8.2           | Framework de tests unitaris per al backend Python.                                               |

### 5.3 Organització del Codi Font

El repositori [18] s'organitza de la manera següent per reflectir clarament la separació entre Cloud Run i Cloud Functions:

```

SM-MiniProjecte2026/
├── backend/                # Cloud Run: FastAPI + pipeline de visió
│   └── src/
│       ├── main.py        # Endpoints /process, /process/voice, /process/text
│       ├── vision.py      # Canny, TSP, Gemini
│       ├── speech.py      # Wrapper Cloud Speech/TTS
│       ├── db.py          # CRUD Firestore per usuari
│       └── storage.py     # Helpers Cloud Storage
├── cloudfunctions/        # Cloud Functions: operacions lleugeres
│   ├── history/           # GET/DELETE historial
│   ├── upload/            # POST pujada d'imatge
│   └── speech/             # Transcripció, TTS i traducció
├── frontend/              # Next.js 16 - interfície web
│   └── app/
│       └── lib/
│           ├── firebase.ts # Init Firebase + signInAnon
│           ├── api.ts      # Client API centralitzat
│           └── styleExtractor.ts # extractStyle() en TypeScript

```

**Listing 2:** Estructura del repositori (simplificada).

## 6 Problemes i Solucions

Durant el desenvolupament del projecte hem trobat diversos problemes tècnics. En aquest apartat documentem els més significatius, la causa arrel i la solució adoptada.

### Problema 1 — Firebase *auth/invalid-api-key* durant el build de producció

**Síntoma.** El build de Docker fallava amb l'error Firebase: Error (auth/invalid-api-key) durant la fase de compilació del frontend Next.js. El build resultava en un codi d'error 1.

**Causa.** Next.js realitza un pre-render estàtic de totes les pàgines durant el build (*Static Site Generation*). El mòdul `firebase.ts` cridava `initializeApp()` a nivell de mòdul (fora de qualsevol funció), de manera que s'executava durant el pre-render. En aquell moment, les variables d'entorn `NEXT_PUBLIC_FIREBASE_*` no estaven disponibles (el fitxer `.env` s'exclou del context de Docker per seguretat), i Firebase rebria una clau d'API buida.

**Solució.** Es va reimplementar `firebase.ts` amb **inicialització diferida** (*lazy initialization*): `initializeApp()` s'executa dins d'una funció getter (`getFirebaseAuth()`) que es crida per primera vegada des d'un `useEffect` del navegador, mai durant el pre-render. A més, es va afegir `auth.authStateReady()` per esperar que Firebase restauri l'estat d'autenticació del `localStorage` abans de fer cap petició autenticada. Es va crear el fitxer `frontend/.env.production` (comès al repositori, ja que conté valors públics) amb les variables necessàries.

### Problema 2 — Variable `SHORT_SHA` buida en desplegament manual

**Síntoma.** En executar `gcloud builds submit` manualment des de la línia de comandes, el build fallava amb l'error «invalid image name: could not parse reference» perquè el nom de la imatge Docker incloïa una etiqueta buida (`:`).

**Causa.** El fitxer `cloudbuild.yaml` utilitza la variable `SHORT_SHA` (hash curt del commit git) per etiquetar les imatges Docker. Aquesta variable s'injecta automàticament quan Cloud

Build s'activa des d'un *trigger* de repositori, però no s'injecta quan es fa un `gcloud builds submit manual`.

**Solució.** Passar la substitució explícitament en cada execució manual:

```
gcloud builds submit --config cloudbuild.yaml \
  --substitutions=SHORT_SHA=$(git rev-parse --short HEAD)
```

### Problema 3 — Error 403 en el desplegament de Cloud Functions

**Síntoma.** El primer desplegament d'una Cloud Function fallava indicant que el service agent no tenia els permisos `artifactregistry.repositories.list` i `artifactregistry.repositories.get`.

**Causa.** Cloud Functions de 2a generació utilitza Artifact Registry per emmagatzemar els contenidors de les funcions. El compte de servei del servei Cloud Functions (`service-PROJECT-NUMBER@gcf-admin-robot.iam.gserviceaccount.com`) no tenia els permisos necessaris sobre Artifact Registry.

**Solució.** Afegir el rol `roles/artifactregistry.reader` al compte de servei:

```
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:service-NUMBER@gcf-admin-robot.iam.gserviceaccount.com" \
  --role="roles/artifactregistry.reader"
```

### Problema 4 — Botó de generació desactivat en producció

**Síntoma.** Desplegada l'aplicació a Cloud Run, el botó «Generar Dibujo» apareixia sempre en gris desactivat. No es produïa cap missatge d'error visible.

**Causa.** El botó es desactiva mentre `uid === null`, és a dir, mentre Firebase no hagi completat el login anònim. La crida a `signInAnonymously()` fallava silenciosament perquè el domini de Cloud Run (`dal-i-api-y55wqasb6a-ew.a.run.app`) no estava a la llista de dominis autoritzats de Firebase Authentication.

**Solució en dues passes.**

1. Afegir el domini al panell de Firebase Console → Authentication → Settings → Authorized domains.
2. Modificar el bloc `catch` de `signInAnon()` per mostrar l'error a la interfície (en comptes de silenciar-lo), facilitant el diagnòstic futur.

### Problema 5 — `auth/admin-restricted-operation`

**Síntoma.** Després de solucionar el problema anterior, l'error canviava a Firebase: `Error (auth/admin-restricted-operation)`.

**Causa.** Tot i tenir el domini autoritzat, el proveïdor d'autenticació anònima no estava habilitat al projecte Firebase.

**Solució.** Activar el proveïdor anònim a Firebase Console → Authentication → Sign-in method → Anonymous → Enable.

### Problema 6 — La Raspberry Pi rebia 404 a `/process/voice`

**Síntoma.** Després de la migració de la parla a Cloud Functions, la Raspberry Pi deixava de rebre coordenades. La petició retornava codi 404.

**Causa.** Durant la migració a Cloud Functions, els endpoints `/process/voice` i `/process/text` es van eliminar del Cloud Run perquè el frontend web ja no els necessitava (orquestraria les crides a la CF directament). Però la Raspberry Pi continuava cridant l'endpoint original del Cloud Run.

**Solució.** Restaurar `/process/voice` i `/process/text` al Cloud Run, afegint una dependència d'autenticació opcional (`_optional_firebase_token`) que retorna `uid="device"` quan no hi ha token, en lloc de retornar error 401. D'aquesta manera els dos clients (navegador i Pi) coexisteixen sense canvis al codi de la Pi.

### Problema 7 — Context de Docker de 364 MB

**Síntoma.** Cada `gcloud builds submit` pujava 364 MB de context al núvol, alentint el procés i consumint quota de Cloud Storage innecessàriament.

**Causa.** El directori de treball incloïa `frontend/node_modules/` (uns 300 MB) en el context de build, tot i existir un fitxer `.gcloudignore`.

**Solució.** Verificar i completar el fitxer `.gcloudignore` per excloure explícitament `node_modules/`, `frontend/.next/`, `frontend/out/`, `backend/.env/` i els directoris `docs/`.

## 7 Resultats

### 7.1 Sistema en Producció

El sistema Dal-i es troba desplegat i accessible públicament a:

<https://dal-i-api-y55wqasb6a-ew.a.run.app>

L'aplicació és funcional en els tres modes d'entrada (selecció, text i veu), amb suport per a 10 idiomes i el Mode Avançat activable. Les Cloud Functions responen de manera independent i el historial per usuari es manté entre sessions.

### 7.2 Pipeline de Visió

El pipeline complet (sense Mode Avançat) processa una imatge típica de 768×768 píxels en **2–5 segons**, depenent del nombre de contorns detectats. Amb el Mode Avançat activat (crida a Gemini), el temps total augmenta fins als **10–20 segons** segons la complexitat de la imatge i la disponibilitat del model.

El sistema genera típicament entre **50 i 500 traços** per imatge, cadascun representat com una seqüència de punts  $\{x, y\}$ . L'optimització TSP (Greedy + 2-opt) redueix el desplaçament en buit del robot entre un 20% i un 40% respecte a un ordre aleatori dels contorns.

### 7.3 Exemples de Resultats

La **Figura 3** i la **Figura 4** mostren exemples de processament amb el sistema. Cada parell mostra la imatge d'entrada (esquerra) i el resultat del sistema (dreta), que representa els

traços que el robot executarà.



(a) Imatge d'entrada (Akagi)



(b) Trazos SVG generats (sense Mode Avançat)

**Figura 3:** Personatge Akagi — processament bàsic, sense IA generativa: només detecció de contorns i vectors SVG.



(a) Imatge d'entrada (Koharu)



(b) Resultat amb Mode Avançat (estil Van Gogh)

**Figura 4:** Personatge Koharu — Mode Avançat activat: Gemini 2.5 Flash aplica l'estil Van Gogh abans de la detecció de contorns.

## 7.4 Tests

El projecte inclou una suite de tests unitaris per al backend Python, situada a `backend/tests/`. Els tests cobreixen els endpoints principals del Cloud Run (`/health`, `/process`, `/process/voice`, `/process/text`) i els mòduls de visió, parla i emmagatzematge. En el moment de tancament del projecte, **34 tests passen** sense errors amb pytest.

La dependència d'autenticació Firebase s'injecta com a *mock* als tests mitjançant `app.dependency_overrides[_verify_firebase_token] = lambda: "test-uid"`, de manera que els tests no necessiten un token real de Firebase.

## 7.5 Infraestructura Desplegada

La **Taula 3** recull tots els recursos GCP actius en el moment de tancament del projecte.

**Taula 3:** Resum dels recursos GCP desplegats.

| Recurs         | Nom          | Regió        | Estat |
|----------------|--------------|--------------|-------|
| Cloud Run      | dal-i-api    | europa-west1 | Actiu |
| Cloud Function | history      | us-central1  | Actiu |
| Cloud Function | upload       | us-central1  | Actiu |
| Cloud Function | speech       | us-central1  | Actiu |
| Cloud Storage  | dal-i-bucket | Multirègió   | Actiu |
| Firestore      | —            | us-central1  | Actiu |
| Firebase Auth  | —            | Global       | Actiu |

## 8 Conclusions i Treball Futur

### 8.1 Conclusions

El projecte Dal-i ha assolit tots els objectius plantejats inicialment. S'ha dissenyat i implementat un sistema complet que cobreix des de la interfície d'usuari fins al maquinari robòtic, passant per un pipeline de visió per computador, models d'IA generativa i una infraestructura *cloud-native* escalable.

Les conclusions principals del projecte són les següents:

**Arquitectura de microserveis.** La separació entre Cloud Run (pipeline de visió, computacionalment intens) i Cloud Functions (operacions lleugeres de dades i parla) ha demostrat ser una decisió encertada. Permet escalar cada component de forma independent i reduir el cost quan el sistema està en repòs, ja que les Cloud Functions no generen cost si no hi ha invocacions.

**Firebase Anonymous Auth.** L'autenticació anònima ha resultat ser una solució elegant per oferir un historial persistent per sessió sense forçar l'usuari a registrar-se. L'UID estable permet associar les dades a un navegador concret sense necessitar cap formulari de registre.

**Traducció automàtica interna.** Processar sempre en anglès internament i traduir les entrades i sortides automàticament ha simplificat enormement el codi de processament d'estils, que no necessita conèixer cap idioma específic. El resultat és un sistema multilingüe robust amb un sobrecost de latència mínim.

**Optimització TSP per a robòtica.** L'aplicació de tècniques d'optimització de rutes (Greedy Nearest Neighbour + 2-opt) al problema d'ordenació de contorns per al robot ha tingut un impacte positiu mesurable en el temps de dibuix. Aquesta connexió entre algorísmica clàssica i robòtica ha estat un dels aspectes més enriquidors del projecte.

**Integració de maquinari.** La coexistència de clients diversos (navegador web amb Firebase Auth i Raspberry Pi sense autenticació) s'ha resolt de manera neta amb la dependència d'autenticació opcional, sense duplicar cap lògica de negoci.

## 8.2 Aprenentatges

El projecte ha permès adquirir experiència pràctica en:

- Desplegament de contenidors Docker multi-etapa a Google Cloud Run.
- Implementació i desplegament de Cloud Functions de 2a generació amb Python.
- Configuració d'autenticació Firebase en aplicacions web estàtiques (Next.js static export).
- Integració de múltiples APIs de GCP (Speech-to-Text, TTS, Translation, Vertex AI) en un sol sistema coherent.
- Algorísmica aplicada (TSP) en un context de robòtica real.
- Gestió de problemes de CORS, IAM i dominis autoritzats en entorns de producció.

## 8.3 Treball Futur

Com a línies de treball futur, proposem les extensions següents:

1. **Generació de vídeo del procés de dibuix.** Capturar la trajectòria del robot en temps real i generar un vídeo time-lapse del dibuix, que es podria compartir directament des de la interfície.
2. **Col·laboració en temps real.** Permetre que múltiples usuaris contribueixin a un mateix dibuix simultàniament, amb sincronització via WebSockets o Firebase Realtime Database.
3. **Ampliació d'estils artístics.** Incorporar un mecanisme de *fine-tuning* de Gemini per afegir nous estils artístics de manera dinàmica, sense canvis al codi.
4. **Dashboard d'administració.** Crear un panell de control per monitoritzar l'ús del sistema, els costos de GCP i les estadístiques de dibuixos generats.
5. **Optimització del pipeline amb GPU.** Explorar l'execució del pipeline de visió en una instància amb GPU a Cloud Run per accelerar el processament de Mode Avançat.
6. **Exportació en formats estàndard.** Permetre descarregar les trajectòries generades en formats industrials com G-code o SVG optimitzat, compatibles amb una gamma més àmplia de màquines CNC i plotters.

## Referències

- [1] Google LLC. *Cloud Speech-to-Text Documentation*. 2024. URL: <https://cloud.google.com/speech-to-text/docs> (cons. 7 de juny de 2026).
- [2] Google LLC. *Vertex AI — Gemini API Documentation*. 2024. URL: <https://cloud.google.com/vertex-ai/generative-ai/docs> (cons. 7 de juny de 2026).
- [3] Google LLC. *Cloud Text-to-Speech Documentation*. 2024. URL: <https://cloud.google.com/text-to-speech/docs> (cons. 7 de juny de 2026).
- [4] Google LLC. *Cloud Translation API Documentation*. 2024. URL: <https://cloud.google.com/translate/docs> (cons. 7 de juny de 2026).
- [5] Google LLC. *Firebase Authentication — Anonymous Sign-In*. 2024. URL: <https://firebase.google.com/docs/auth/web/anonymous-auth> (cons. 7 de juny de 2026).
- [6] Google LLC. *Firestore Documentation*. 2024. URL: <https://cloud.google.com/firestore/docs> (cons. 7 de juny de 2026).
- [7] Google LLC. *Cloud Storage Documentation*. 2024. URL: <https://cloud.google.com/storage/docs> (cons. 7 de juny de 2026).
- [8] Vercel Inc. *Next.js Documentation — The React Framework for the Web*. 2024. URL: <https://nextjs.org/docs> (cons. 7 de juny de 2026).
- [9] Meta Open Source. *React — The Library for Web and Native User Interfaces*. 2024. URL: <https://react.dev> (cons. 7 de juny de 2026).
- [10] Google LLC. *Cloud Run Documentation*. 2024. URL: <https://cloud.google.com/run/docs> (cons. 7 de juny de 2026).
- [11] Google LLC. *Cloud Functions Documentation (2nd Generation)*. 2024. URL: <https://cloud.google.com/functions/docs> (cons. 7 de juny de 2026).
- [12] Tailwind Labs Inc. *Tailwind CSS Documentation*. 2024. URL: <https://tailwindcss.com/docs> (cons. 7 de juny de 2026).
- [13] Sebastián Ramírez. *FastAPI — Modern, Fast Web Framework for Building APIs with Python*. 2024. URL: <https://fastapi.tiangolo.com> (cons. 7 de juny de 2026).
- [14] John Canny. “A Computational Approach to Edge Detection”. A: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 8.6 (1986), pàg. 679–698. DOI: [10.1109/TPAMI.1986.4767851](https://doi.org/10.1109/TPAMI.1986.4767851).
- [15] Gary Bradski. “The OpenCV Library”. A: *Dr. Dobb’s Journal of Software Tools* (2000). URL: <https://opencv.org> (cons. 7 de juny de 2026).
- [16] David L. Applegate, Robert E. Bixby, Vašek Chvátal i William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006. ISBN: 978-0-691-12993-8.
- [17] Charles R. Harris et al. “Array programming with NumPy”. A: *Nature* 585 (2020), pàg. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [18] Grup G4-6, UAB Sistemes Multimèdia. *Dal-i: Sistema Robòtic de Dibuix Col·laboratiu*. Repositori del codi font del projecte. 2026. URL: <https://github.com/moisan7/SM-MiniProjecte2026> (cons. 7 de juny de 2026).